ML Question 01:

1. Write a Python program to find common items from two lists.
2. Write a Python program to print the numbers of a specified list after removin g even numbers from it using loop comprehensions:
3. Write a code to have this output:

   *

   * *

   * * *

   * * * *

   * * * * *

4. Write a Python function that takes two lists and returns True if they have at least one common member.
5. Write a Python program that create a dictionary and print its element like this:

   #output is:
   #Key1>Value1
   #Key2>Value2
   #Key3>Value3……

Solution:

1. Write a Python program to find common items from two lists.

```
list1 = [1, 2, 3, 4, 5]
list2 = [3, 4, 5, 6, 7]

common = []

for item in list1:
    if item in list2:
        common.append(item)

print("Common items:", common)
```

2. Write a Python program to print the numbers of a specified list after removing even numbers from it using list comprehensions.

```
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9]

odd_numbers = [x for x in numbers if x % 2 != 0]

print(odd_numbers)
```

3. Write a code to have this output:

```
*
* *
* * *
* * * *
* * * * *
for i in range(1, 6):
    for j in range(i):
        print("*", end=" ")
    print()
```

4. Write a Python function that takes two lists and returns True if they have at least one common member.

```
def common_member(list1, list2):
    for item in list1:
        if item in list2:
            return True
    return False

list1 = [1, 2, 3]
list2 = [4, 5, 3]

print(common_member(list1, list2))
```

5. Write a Python program that creates a dictionary and prints its elements like this:

Output:

Key1 > Value1

Key2 > Value2

Key3 > Value3

```
my_dict = {
    "Key1": "Value1",
    "Key2": "Value2",
    "Key3": "Value3"
}

for key, value in my_dict.items():
    print(key, ">", value)
```

ML Question 02:

Exercise 2: Exploratory Data Analysis (EDA) Exercise Using Pandas

Assume you have a CSV dataset called **data.csv** that contains numeric and categorical columns such as:

- "Age"
- "Salary"
- "Department"
- "Score"
- "Result" (label column)

PART 1 — Load the dataset

Write Python code to:

- Import pandas
- Load the CSV file "data.csv" into a DataFrame named df
- Print the **first 5 rows**
- Print the **last 5 rows**

PART 2 — Null value analysis

Using pandas:

- Print the **total number of null cells** in the entire DataFrame
- Print the **number of null cells per column**

PART 3 — Handling missing values

Perform the following:

- **Drop** all rows where the column "Salary" is null

- **Fill** null values in the column "Department" using its **mode**
- **Fill** null values in "Age" using the **median**

PART 4 — Frame shape and slicing

Do the following:

- Print the **number of rows and columns** in the DataFrame
- Split the DataFrame into:
  - **X** → all columns **except** "Result"
  - **y** → the column "Result"
- Print rows from index **10 to 20** (inclusive)

PART 5 — Duplicate handling

Write code to:

- Check for duplicates based on the columns ["Age", "Salary"]
- Remove these duplicates

PART 6 — Plotting with matplotlib

Using pandas + matplotlib:

- Create a scatter plot between "Age" and "Salary"
- Create another scatter plot between "Age" and "Score"
- Create a **correlation scatter plot** between "Salary" and "Score"

Exercise 3: ML Algorithm

Complete the following Python script to create and evaluate a model that predicts **monthly product sales** based on advertising spending on TV, Radio, and Social Media.
Fill in the missing code parts where indicated.

```
import pandas as pd
```

```
# Dataset
dataset = pd.DataFrame({
    'Month': list(range(1, 13)),
    'TV_Ads': [12000, 15000, 8000, 6000, 9000, 14000, 16000, 11000, 7000, 13000, 15500, 5000],
    'Radio_Ads': [4000, 5000, 3000, 2000, 3000, 4500, 4800, 3500, 2300, 4200, 5200, 1500],
    'Social_Media_Ads': [2000, 3500, 1200, 800, 1500, 2500, 3000, 1800, 1000, 2600, 3400, 600],
    'Monthly_Sales': [220, 250, 180, 150, 170, 230, 260, 200, 160, 240, 255, 140]
})
```

a) Print the dataset and check its dimensions.

Fill in the missing code:

```
print("Dataset:")
# TODO: print the dataset
print("\nShape of dataset:")
# TODO: print the shape
```

b) Select suitable input features (X) and the output variable (y), excluding unnecessary columns.

Month is not useful as a feature.

```
# TODO: Select X (all ad columns)
# TODO: Select y (Monthly_Sales)
```

c) Split the data into training and testing sets (test size = 20%).

```
from sklearn.model_selection import train_test_split
```

```
# TODO: perform train-test split here
```

d) Create and train a machine learning model using a suitable algorithm.

Use **LinearRegression** or another valid regression model.

```
from sklearn.linear_model import LinearRegression

# TODO: create the model

# TODO: train the model
```

e) Use the trained model to make and print predictions on the test data.

```
# TODO: make predictions

# TODO: print predictions
```

f) Evaluate the model using MAE, MSE, and R^2 Score.

from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

```
# TODO: compute MAE

# TODO: compute MSE

# TODO: compute R2 score


# Print results
```

Exercise 4: ML Algorithm

Predicting Loan Approval (Yes/No)

You are given a dataset of applicants with income, credit score, and years of employment. Your task is to **predict whether a loan will be approved**.

```
import pandas as pd

# Dataset

dataset = pd.DataFrame({

  'Applicant_ID': list(range(1, 13)),

  'Income_k': [50, 80, 30, 25, 40, 60, 75, 55, 35, 65, 70, 28],

  'Credit_Score': [750, 800, 680, 600, 620, 720, 790, 710, 640, 730, 770, 610],

  'Years_Employed': [10, 12, 3, 1, 2, 8, 11, 6, 4, 9, 10, 1],

  'Loan_Approved': ['Yes','Yes','No','No','No','Yes','Yes','Yes','No','Yes','Yes','No']

})
```

Exercise Steps

a) Print the dataset and its shape.

b) Select input features (X) and target (y)

- Features: Income_k, Credit_Score, Years_Employed
- Target: Loan_Approved

c) Split dataset into training and testing sets (test size 20%).

d) Train a classification model

- Example: LogisticRegression

e) Make predictions on test data and print them.

f) Evaluate the model using

- Confusion Matrix
- F1-score
- Accuracy (optional)

```
Ex2 Solution:
# ==========================
# PART 1 — Load the dataset
# ==========================

import pandas as pd
import matplotlib.pyplot as plt

# Load CSV file
df = pd.read_csv("data.csv")

# First 5 rows
print(df.head())

# Last 5 rows
print(df.tail())


# ==========================
# PART 2 — Null value analysis
# ==========================

# Total number of null cells
print("Total Null Values:", df.isnull().sum().sum())

# Null values per column
print(df.isnull().sum())


# ==========================
```

```
# PART 3 — Handling missing values
# ==========================

# Drop rows where Salary is null
df = df.dropna(subset=["Salary"])

# Fill null values in Department with mode
df["Department"] =
df["Department"].fillna(df["Department"].mode()[0])

# Fill null values in Age with median
df["Age"] = df["Age"].fillna(df["Age"].median())


# ==========================
# PART 4 — Frame shape and slicing
# ==========================

# Print number of rows and columns
print("Shape:", df.shape)

# X -> all columns except Result
X = df.drop("Result", axis=1)

# y -> Result column
y = df["Result"]

# Print rows from index 10 to 20 (inclusive)
print(df.loc[10:20])


# ==========================
```

```
# PART 5 — Duplicate handling
# ==========================

# Check duplicates based on Age and Salary
print(df.duplicated(subset=["Age", "Salary"]))

# Remove duplicates
df = df.drop_duplicates(subset=["Age", "Salary"])


# ==========================
# PART 6 — Plotting with matplotlib
# ==========================

# Scatter plot: Age vs Salary
plt.figure(figsize=(6,4))
plt.scatter(df["Age"], df["Salary"])
plt.title("Age vs Salary")
plt.xlabel("Age")
plt.ylabel("Salary")
plt.show()

# Scatter plot: Age vs Score
plt.figure(figsize=(6,4))
plt.scatter(df["Age"], df["Score"])
plt.title("Age vs Score")
plt.xlabel("Age")
plt.ylabel("Score")
plt.show()

# Scatter plot: Salary vs Score
plt.figure(figsize=(6,4))
plt.scatter(df["Salary"], df["Score"])
plt.title("Salary vs Score")
plt.xlabel("Salary")
plt.ylabel("Score")
plt.show()
```

ML Question 03:

Model Selection & Problem Understanding (Conceptual)

You are given the following real-world problems.
For each problem:

1. Choose the **most suitable algorithm**
2. Justify your choice (2–3 sentences)
3. Mention **one disadvantage** of the chosen model

Problems:

a) Predicting **house prices** based on size, location, and number of rooms

1) **Linear regression:**
2) **Pros: Simple, interpretable, fast to train.**
3) **Cons: Assumes Linear Relation ships, cannot capture complex patterns**

b) Detecting **spam emails** (spam / not spam)

1) **SVM**
2) **Pros: high accuracy with proper tuning. Robust to overfitting.**
3) **Cons:slower to train on large datasets.**

c) Classifying handwritten digits (0–9)

1) **SVM**
2) **Pros: strong performance with less data. High Accuracy.**
3) **Cons: Does not consider pixel position and relations to other pixels.**

d) Customer segmentation based on annual income and spending behavior

1) **K-means**
2) **Fast, scalable and easy to implement**
3) **Sensitive to outliers. Requires scaling. Requires a pre-defined k value**

e) Medical diagnosis with **small dataset and independent features**

1) **Logistic Regression**
2) **Pros: Works well with small datasets (low variance), interpretable**
3) **Cons: sensitive to outliers. Considers linear relations.**

ML Question 04:

Linear & Logistic Regression (Math + Concept)

Part A – Linear Regression

Given the model:

$y=2x+5$

1. What does the **slope** represent?
2. What does the **intercept** represent?
3. What happens if we **add irrelevant features**?

Solution:

1. The **slope (2)** represents the **rate of change** of the dependent variable y with respect to the independent variable x. **Example**: if x is hours studied and y is the test score, then studying one more hour is associated with a **2-point increase** in score.
2. The intercept represents the value of y when x = 0. **Example**: the student who studies 0 hours is predicted to score 5.
3. Adding irrelevant features, means adding noise. Usually noise leads to **overfitting** and **high variance**.

Part B – Logistic Regression

A logistic regression model outputs:

$P(y=1 \mid x)=0.72$

1. What does this value mean?
2. What threshold is commonly used for classification?
3. How would changing the threshold affect **precision and recall**?

Solution:

1. This means that, given the input features x, the model estimates a **72% probability** that the label y belongs to class 1 (the positive class)

   Note: logistic regression output probabilities, not hard classifications, this allows for precise decision making based on the type of problem.

MLQuestion05:

Decision Trees & Random Forests (Reasoning)

Part A

Explain **why decision trees are prone to overfitting**.

Part B

How does **Random Forest** reduce overfitting?
Mention **two mechanisms**.

Part C

Which model would you prefer for:

- Small dataset?
- Large dataset with noisy features?

Justify briefly.

Solution:

Part A:

1. They keep splitting until pure leaves (by default).
    a. Most implementations, including scikit-learn) split nodes until every leaf is pure (all samples in a leaf belong to the same class) or until each leaf node has only one sample.
    b. **Result**: the tree memorizes noise, outliers or any special information within the training data. This leads to **perfect training accuracy but poor generalization**.
2. High Variance (sensitive to small data changes)
    a. A tiny change in training set (eg. Removing one sample) can lead to a **completely different tree structure**.
    b. This instability => high variance => overfitting.
3. No Built-in Regularization:
    a. Unlike linear models (eg. Logistic regression with L2 penalty), a raw decision tree has **no mechanism to penalize complexity**.
    b. It will happily create deep, complex ruls for rare patterns that won't repeat new data.
4. They model noise as if it were signal

In practical terms: if this were a medical test predicting disease (where y=1 means that the patient has the disease), the model says that 72% chance the patient has the disease.

2. The default and most common threshold is 0.5.

   Rule:

   If P(y = 1 | x) >= 0.5 ==> predict class 1

   If P(y = 1 | x) < 0.5 ==> predict class 0

   In our case 0.72 > 0.5, which means that the model has classified the instance x as class 1.

3. Remember the formulas of Recall and Precision:

   Recall = $\frac{TP}{TP+FN}$: Lowering threshold catches more actual positives (FN decrease ==> recall increase)

   Precision = $\frac{TP}{TP+FP}$: Raising the threshold ensures only high-confidence predictions are made (FP decrease ==> precision increase).

Example: in **disease screening**, you might **lower the threshold** to maximize recall (catch All sick patients, even it means more false alarms)

In **spam prediction**, you might **raise the threshold** to maximize precision (avoid making Important emails as spam).

a. If a feature has a **spurious (مزيف) correlation** with the target in the training set (due to randomness), the tree split on it, especially if it reduces impurity slightly.
b. With enough depth, the tree **fits random fluctuations (التقلبات)** as **rules**.

Enhancements to decision trees to avoid overfitting.

Technique	Effect
Limit max_depth	Stops the tree from growing too deep (e.g., depth ≤ 5)
Set min_samples_split	Requires a minimum number of samples to split a node (e.g., ≥10)
Set min_samples_leaf	Ensures each leaf has enough samples (e.g., ≥5)
Pruning (post training)	Remove branches that don't improve validation performance

Example: Overfit VS Regularized Tree

- **Overfit Tree**
  - Depth = 20
  - Accuracy on train = 100%
  - Accuracy on test = 68%
- **Regularized Tree**
  - max_depth=4, min_samples_leaf=10
  - Accuracy on train = 85%
  - Accuracy on test = 83%

Part B:

Random Forest reduces overfitting by combining two key ideas from ensemble learning: **bagging (bootstrap aggregating)** and **random feature selection**. Together, they **decrease variance** which is the main cause of overfitting in decision trees, while **preserving or even improving predictive accuracy**.

1. **Bagging (Bootstrap Aggregating)**
    a. Random Forest trains **many decision trees** (eg. 100 or 500), each on a **random subset of the training data**.
    b. Each subset is created by **sampling with replacement** (bootstrap sample), so some observatios are repeated, and ~37% are left out (**out-of-bag samples**).
    c. **Why it helps:**
        i. Averages predictions across many trees => **reduces variance.**
        ii. No single tree sees the full dataset => **less chance to memorize noise.**
        iii. Out-of-bag samples provide an **unbiased estimate of model performance**. In other words, samples that are not selected, wil be used as a validation set for each model.
2. **Random Feature Selection at Each Split**
    a. In a standard decision tree, the best split is chosen by evaluating all features.
    b. In Random Forest, at each split, only random subset of features (eg. $\sqrt{p}$ for classification, p/3 for regression, where p = total features) is considered.
    c. **Why it helps:**
        i. **Reduces correlation between trees**: if all trees used the same dominant features, the would make similar errors. Random feature selection forces diversity.
        ii. **Prevents a few strong features from dominating** every tree.
        iii. **Increase ensemble robustness**: Even if some features are noisy or irrelevant, the forest isn't overly reliant on them.

ML Question 06:

Question 1:

Convert the following FOL sentences into CNF:

Note 1: There is no need to standardize/name variables apart.
Note 2: You should specify the name and details of each step.

1- $\forall x [\forall y\, C(y) \Rightarrow P(x, y)] \Rightarrow [\exists z\, P(z, x)]$

Eliminate implication
$\forall x [\forall y \neg C(y) \vee P(x, y)] \Rightarrow [\exists z\, P(z, x)]$
$\forall x [\neg \forall y \neg C(y) \vee P(x, y)] \vee [\exists z\, P(z, x)]$
Drive in negation
$\forall x [\exists y\, C(y) \wedge \neg P(x, y)] \vee [\exists z\, P(z, x)]$
Skolemize
$\forall x [C(F(x)) \wedge \neg P(x, F(x))] \vee P(G(x), x)$
Drop universal quantifiers
$[C(F(x)) \wedge \neg P(x, F(x))] \vee P(G(x), x)$
Distribute $\wedge$ over $\vee$
$[C(F(x)) \vee P(G(x), x)] \wedge [\neg P(x, F(x)) \vee P(G(x), x)]$

2- $\exists x \forall y\, H(x, y)$

Skolemize
$\forall y\, H(Peter, y)$
Drop universal quantifiers
$H(Peter, y)$

3- $\forall y \exists x\, L(x, y)$

Skolemize
$\forall y\, L(F(y), y)$
Drop universal quantifiers
$L(F(y), y)$

4- $\forall z\, A(z) \Rightarrow [\forall t\, B(t) \Rightarrow \neg C(z, t)]$

Eliminate implication
$\forall z\, \neg A(z) \vee [\forall t\, B(t) \Rightarrow \neg C(z, t)]$
$\forall z\, \neg A(z) \vee [\forall t\, \neg B(t) \vee \neg C(z, t)]$
Drop universal quantifiers
$\neg A(z) \vee \neg B(t) \vee \neg C(z, t)$

5- $\forall w\, [\exists x\, Pop(x) \wedge Lu\,(w, x)] \Rightarrow S(w)$

Eliminate implication
$\forall w\, [\neg \exists x\, Pop(x) \wedge Lu\,(w, x)] \vee S(w)$
Drive in negation
$\forall w\, \forall x [\neg (Pop(x) \wedge Lu\,(w, x))] \vee S(w)$
$\forall w\, \forall x \neg Pop(x) \vee \neg Lu\,(w, x)) \vee S(w)$
Drop universal quantifiers
$\neg Pop(x) \vee \neg Lu\,(w, x)) \vee S(w)$

6- $[Bcc(G(x)) \wedge \neg Cc(x, G(x))] \vee Cc\,(H(x), x)$

Distribute $\wedge$ over $\vee$
$[Bcc(G(x)) \vee Cc\,(H(x), x)] \wedge [\neg Cc(x, G(x)) \vee Cc\,(H(x), x)]$

ML Question 07 :

Naive Bayes (Conceptual + Intuition)

1. What is the **"naive" assumption** in Naive Bayes?
2. Why does Naive Bayes work well for **text classification**?
3. When does Naive Bayes perform poorly?

Solution:

Naive Bayes (Conceptual + Intuition)

1. What is the "naive" assumption in Naive Bayes?

The "naive" assumption is that **all input features are conditionally independent given the class label**.

In other words:

Knowing the value of one feature tells you nothing about the value of any other feature, as long as you already know the class.

Mathematically:
For features $x_1, x_2, \ldots, x_n$ and class y, Naive Bayes assumes:

$$P(x_1, x_2, \ldots, x_n \mid y) = P(x_1 \mid y) \cdot P(x_2 \mid y) \cdot \ldots \cdot P(x_n \mid y)$$

Intuition:

- This assumption is rarely true in real-world data (e.g., words like "machine" and "learning" often co-occur).
- Despite being unrealistic, the model often performs well in practice—which is why it's called "naive" but effective.

2. Why does Naive Bayes work well for text classification?

Naive Bayes is particularly effective for text tasks (e.g., spam detection, sentiment analysis) for several reasons:

- **Handles high-dimensional, sparse data**: Text represented as bag-of-words or TF-IDF often has thousands of features. Naive Bayes scales efficiently by using simple frequency counts.
- **Data-efficient**: Requires relatively little training data; probabilities are estimated from word counts per class.

- **Independence assumption is "good enough"**: While words are correlated, the dominant signal in text classification usually comes from **individual word frequencies per class** (e.g., "free" appears more in spam emails).
- **Fast and interpretable**: Training and prediction are computationally cheap. You can inspect which words most strongly influence a prediction.

Example: In spam filtering, words like "win", "free", and "cash" are strong individual indicators of spam—even if they appear together, treating them independently still captures sufficient predictive power.

3. When does Naive Bayes perform poorly?

Despite its strengths, Naive Bayes has limitations and may underperform in the following situations:

- **Strong feature dependencies matter**: If the interaction between features is critical (e.g., "fever + cough" together indicate flu, but neither alone is sufficient), Naive Bayes will miss this.
- **Poor probability calibration**: It often produces overconfident probability estimates (e.g., predicts 99% when the true likelihood is 70%), making it unsuitable for applications requiring well-calibrated risks.
- **Unseen features in test data**: If a word appears in the test set but was never seen in the training data for a given class, its probability becomes zero—causing the entire prediction to collapse to zero.
- **Non-Gaussian continuous features**: Gaussian Naive Bayes assumes each continuous feature follows a normal distribution within each class. Performance degrades if this assumption is violated (e.g., skewed or multi-modal data).
- **Complex decision boundaries**: Naive Bayes induces relatively simple decision surfaces and cannot model intricate, non-linear patterns that tree-based or neural models can capture.

ML Question 08 :

Unsupervised Learning – K-Means

You apply K-Means with **k = 3** to a dataset.

1. What does each cluster represent?
2. What happens if:
   a. k is too small?
   b. k is too large?
3. Why is **initialization** important in K-Means?

Solution:

1. What does each cluster represent?

Each cluster represents a **group of data points that are more similar to each other (in terms of feature values) than to points in other clusters**.

- K-Means partitions the data into **3 disjoint groups** by minimizing the **within-cluster sum of squared distances** (inertia) to their respective centroids.
- The **centroid** of each cluster (the mean of all points in the cluster) acts as the "prototype" or representative of that group.
- **Interpretation depends on context**:
  - In customer segmentation: Cluster 1 = "High-value customers", Cluster 2 = "Bargain shoppers", Cluster 3 = "Inactive users".
  - In image compression: Each cluster represents a dominant color.
  - In biology: Each cluster may correspond to a distinct species or cell type.

Note: Since K-Means is unsupervised, clusters have **no pre-defined meaning**—their interpretation requires domain knowledge.

2. What happens if:

a. *k* is too small?

- Distinct groups are forced into the same cluster.
- **High within-cluster variance**: Each cluster becomes too broad and heterogeneous.
- **Loss of meaningful patterns**: Important subgroups are masked (e.g., treating both "premium" and "budget" customers as one group).
- **Result**: Model fails to capture the true structure of the data → **high bias**.

b. k is too large?

- Natural groups are split into artificial sub-clusters.
- **Clusters may represent noise or outliers** rather than real patterns.
- **Poor generalization**: The model fits random fluctuations in the data.
- **Result**: Reduced interpretability and instability across runs → **high variance** (a form of overfitting in unsupervised learning).

Goal: Choose k that balances **cohesion** (tight clusters) and **separation** (distinct clusters), often using the **Elbow Method** or **Silhouette Score**.

3. Why is initialization important in K-Means?

Initialization determines the **starting positions of the k centroids**—and K-Means is **sensitive to this choice** because:

- **Poor initialization** (e.g., centroids starting very close together) can lead to:
  - Suboptimal clustering (higher inertia),
  - Empty or meaningless clusters,
  - Inconsistent results across runs.

Solution: Use **k-means++** initialization (default in scikit-learn), which:

- Spreads initial centroids apart intelligently,
- Leads to faster convergence and more stable, higher-quality clusters.

Best practice: Run K-Means multiple times with different random seeds (or use `n_init` in scikit-learn) and keep the best result based on inertia.

ML Question 09:

Performance Metrics (Very Important)

A binary classifier produces the following confusion matrix:

	Predicted +	Predicted -
Actual +	40	10
Actual -	5	45

1. Compute:
    a. Accuracy
    b. Precision
    c. Recall
    d. F1-score
2. Which metric is most important in:
    a. Cancer detection?
    b. Spam detection?
    c. Explain.

Solution:

Given Confusion Matrix:

Actual Positive	
Actual Negative	

From the table:

- True Positives (TP) = 40
- False Negatives (FN) = 10
- False Positives (FP) = 5
- True Negatives (TN) = 45

1. Compute:

a. Accuracy

Accuracy = (TP + TN) / (TP + TN + FP + FN)
= (40 + 45) / (40 + 45 + 5 + 10)
= 85 / 100
= **0.85** or **85%**

b. Precision
Precision = TP / (TP + FP)
= 40 / (40 + 5)
= 40 / 45
≈ **0.889** or **88.9%**

c. Recall
Recall = TP / (TP + FN)
= 40 / (40 + 10)
= 40 / 50
= **0.80** or **80%**

d. F1-score
F1 = 2 × (Precision × Recall) / (Precision + Recall)
= 2 × (0.889 × 0.80) / (0.889 + 0.80)
= 2 × (0.7112) / (1.689)
≈ 1.4224 / 1.689
≈ **0.842** or **84.2%**

Summary:

- Accuracy: 85%
- Precision: 88.9%
- Recall: 80%
- F1-score: 84.2%

2. Which metric is most important in:

a. Cancer detection?
Recall is most important.

Why?
In cancer screening, missing a real case (a false negative) can have life-threatening consequences. It's better to flag more patients for additional tests—even if some are false alarms—than to miss someone who actually has cancer. Therefore, **maximizing recall** (minimizing false negatives) is critical.

b. Spam detection?
Precision is most important.

Why?
In email filtering, marking a legitimate email as spam (a false positive) is very problematic—you might cause the user to miss an important message from a friend, employer, or bank. It's more acceptable to let a few spam emails into the inbox (lower recall) if it means **almost all real emails are preserved**. Hence, **high precision** is preferred.

c. Explanation

- **Recall** focuses on correctly identifying *all actual positive cases*. It's vital when **false negatives are dangerous or costly** (e.g., disease diagnosis, fraud detection).
- **Precision** focuses on ensuring that *predicted positives are truly positive*. It's crucial when **false positives cause harm or inconvenience** (e.g., spam filters, recommendation systems).
- **Accuracy alone can be misleading**, especially when classes are imbalanced (e.g., only 1% of emails are spam). Always choose metrics based on the **real-world impact of errors**.

ML Question 10:

Generalization & Model Behavior

Answer briefly:

1. Define **generalization**
2. Difference between:
    a. Overfitting
    b. Underfitting
3. Give **two techniques** to reduce overfitting

Solution:

1. Define generalization
Generalization is the ability of a machine learning model to perform well on **new, unseen data**—not just the data it was trained on. A model with good generalization captures the underlying patterns in the data rather than memorizing the training examples.

2. Difference between:

a. Overfitting
Overfitting occurs when a model **learns the training data too well**, including its noise and outliers. As a result, it achieves very high accuracy on the training set but **performs poorly on new data**. This indicates **high variance** and poor generalization.

b. Underfitting
Underfitting occurs when a model is **too simple to capture the underlying pattern** in the data. It performs poorly on **both the training and test sets**, indicating **high bias** and insufficient learning.

Key distinction:

- Overfitting → **Too complex** → fits noise
- Underfitting → **Too simple** → misses patterns

3. Give two techniques to reduce overfitting

1. **Regularization**
   Add a penalty term to the loss function (e.g., L1/Lasso or L2/Ridge) to discourage overly complex models and shrink feature weights.
2. **Cross-validation**
   Use techniques like k-fold cross-validation to evaluate model performance on

multiple subsets of data, ensuring the model doesn't rely on a specific training split.

Other valid techniques include:

- Increasing training data
- Reducing model complexity (e.g., fewer layers in a neural network, lower tree depth)
- Early stopping (in iterative algorithms)
- Dropout (in neural networks)

ML Question 11:

Normalization vs Standardization

1. Define normalization
2. Define standardization
3. Which algorithms are sensitive to feature scale?
4. Which preprocessing would you apply for:
    a. KNN
    b. SVM
    c. Decision Trees

Solution:

1. Define normalization

Normalization (also called **Min-Max scaling**) rescales features to a fixed range, typically **[0, 1]**.

It is calculated as:

$$Normalized\ value = \frac{x - x_{min}}{x_{max} - x_{min}}$$

This preserves the shape of the original distribution and is useful when you know the boundaries of your data.

2. Define standardization

Standardization (also called **Z-score normalization**) transforms features to have a **mean of 0 and a standard deviation of 1**.

It is calculated as:

$$Standardized\ value = \frac{x - \mu}{\sigma}$$

where μ is the mean and σ is the standard deviation of the feature.

This method does not bound values to a specific range and works well when data follows (or approximates) a Gaussian distribution.

3. Which algorithms are sensitive to feature scale?

Algorithms that rely on **distance measures** or **gradient-based optimization** are sensitive to feature scale, including:

- **K-Nearest Neighbors (KNN)**
- **Support Vector Machines (SVM)**
- **Logistic Regression** (when using regularization)
- **Neural Networks**
- **K-Means Clustering**
- **Principal Component Analysis (PCA)**

These algorithms can be **dominated by features with larger scales**, leading to poor performance if features are not scaled.

Note: Tree-based models (e.g., Decision Trees, Random Forests, Gradient Boosting) are **not sensitive** to feature scale, as they split on one feature at a time.

4. Which preprocessing would you apply for:

a. KNN
Standardization or normalization (either works, but **standardization is often preferred**).
Why? KNN uses distance (e.g., Euclidean), so all features must contribute equally.

b. SVM
Standardization (strongly recommended).
Why? SVM is sensitive to feature scale, especially with RBF or polynomial kernels. Standardization ensures fair contribution from all features.

c. Decision Trees
No scaling needed (neither normalization nor standardization).
Why? Decision trees split on individual features using thresholds, so the scale does not affect the splits.

Summary Table

KNN
SVM
Decision Trees

ML Question 12:

Scenario

A student is training different machine learning models on the **same dataset**.

- Dataset size: **1,000 samples**
- Features: **10 numerical features**
- Task: **Binary classification**

The student tests three models and obtains the following results:

Model	Training Accuracy	Test Accuracy
Model A	98%	55%
Model B	72%	70%
Model C	60%	58%

Questions

Q1. Identify the problem

For each model (A, B, C), choose the correct behavior:

- Overfitting
- Underfitting
- Good generalization

Explain your answer in **one sentence per model**.

Q2. Explain the cause

For **Model A**:

1. Why is the training accuracy very high?
2. Why is the test accuracy low?

Q3. Model Complexity

Which models match to the **most likely algorithm**:

- Linear model (Logistic Regression without regularization)
- Decision Tree with large depth

Justify your choice briefly.

Q4. Improving the Models

For **Model A (overfitting)**, suggest **two solutions**.

For **Model C (underfitting)**, suggest **two solutions**.

Q5. Bias–Variance Perspective

Fill in the table:

Problem	Bias	Variance
Overfitting	?	?
Underfitting	?	?

Solution:

Q1. Identify the problem

For each model, identify its behavior and explain briefly.

- **Model A**: **Overfitting**
  It performs almost perfectly on training data but poorly on test data, indicating it has memorized noise instead of learning general patterns.
- **Model B**: **Good generalization**
  Training and test accuracies are very close (72% vs. 70%), showing the model learned meaningful patterns without overfitting or underfitting.
- **Model C**: **Underfitting**
  It performs poorly on both training and test data, suggesting the model is too simple to capture the underlying relationship.

Q2. Explain the cause (Model A)

1. **Why is the training accuracy very high?**
   Model A is overly complex and has fit the training data almost perfectly—including noise and random fluctuations.
2. **Why is the test accuracy low?**
   Because the model learned patterns specific to the training set that do not apply to new, unseen data, leading to poor generalization.

Q3. Model Complexity – Matching Algorithms

- **Model A (98% train, 55% test) → Decision Tree with large depth**
  A deep decision tree can create overly complex rules that memorize the training data, causing severe overfitting.
- **Model B (72% train, 70% test) → Linear model (Logistic Regression without regularization)**

A linear model is simple and stable, often achieving balanced performance on moderate-sized datasets with numerical features.

Q4. Improving the Models

For Model A (overfitting), suggest two solutions:

1. **Reduce model complexity**: For example, limit the depth of a decision tree or increase *k* in KNN.
2. Use ensemble methods like **Random Forest** with constraints.

For Model C (underfitting), suggest two solutions:

1. **Increase model complexity**: Use a more flexible model (e.g., add polynomial features, use a deeper tree, or switch to a non-linear algorithm).
2. **Perform feature engineering**: Create new informative features or remove irrelevant ones to help the model capture patterns better.

Q5. Bias–Variance Perspective

Overfitting
Underfitting

Explanation:

- **Overfitting**: The model is too complex (low bias) but sensitive to noise (high variance).
- **Underfitting**: The model is too simple (high bias) and fails to capture patterns (low variance).